

Structured of Programming Language

Using Python



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Object Oriented Programming



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

At the end of the module the student is expected to

Understand OOP in Python

Define the following

Class

Inheritance

Operator Overloading

Apply OOP concepts in Python



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Topics

Python OOP

Python class

Python Inheritance

Python Multiple Inheritance

Operator Overloading



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Object Oriented Programming

Python is a multi-paradigm programming language. It supports different programming approaches.

One of the popular approaches to solve a programming problem is by creating objects.

This is known as Object-Oriented Programming (OOP).



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

An object has two characteristics:

- Attributes
- behavior

Let's take an example:

A parrot is can be an object,as it has the following properties:

- name, age, color as attributes
- singing, dancing as behavior



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The concept of OOP in Python focuses on creating reusable code. This concept is also known as DRY (Don't Repeat Yourself).

In Python, the concept of OOP follows some basic principles:

Class

A class is a blueprint for the object.

We can think of class as a sketch of a parrot with labels. It contains all the details about the name, colors, size etc. Based on these descriptions, we can study about the parrot. Here, a parrot is an object.

The example for class of parrot can be :

class Parrot:

Here, we use the class keyword to define an empty class Parrot. From class, we construct instances. An instance is a specific object created from a particular class.

Object

An object (instance) is an instantiation of a class. When class is defined, only the description for the object is defined. Therefore, no memory or storage is allocated.

The example for object of parrot class can be:
obj = Parrot()

Here, obj is an object of class Parrot.

Suppose we have details of parrots. Now, we are going to show how to build the class and objects of parrots.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

```

1 class Parrot:
2     # class attribute
3     species = "bird"
4     # instance attribute
5     def __init__(self, name, age):
6         self.name = name
7         self.age = age
8
9 # instantiate the Parrot class
10 blu = Parrot("Blu", 10)
11 woo = Parrot("Woo", 15)
12
13 # access the class attributes
14 print("Blu is a {}".format(blu.__class__.species))
15 print("Woo is also a {}".format(woo.__class__.species))
16
17 # access the instance attributes
18 print("{} is {} years old".format( blu.name, blu.age))
19 print("{} is {} years old".format( woo.name, woo.age))

```

Program Code

```

Blu is a bird
Woo is also a bird
Blu is 10 years old
Woo is 15 years old

```

Program Output

In the above program, we created a class with the name Parrot. Then, we define attributes. The attributes are a characteristic of an object.

These attributes are defined inside the `__init__` method of the class. It is the initializer method that is first run as soon as the object is created.

Then, we create instances of the Parrot class. Here, blu and woo are references (value) to our new objects.

We can access the class attribute using `__class__.species`. Class attributes are the same for all instances of a class. Similarly, we access the instance attributes using `blu.name` and `blu.age`. However, instance attributes are different for every instance of a class.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Methods

Methods are functions defined inside the body of a class. They are used to define the behaviors of an object.

```
1 class Parrot:
2     # instance attributes
3     def __init__(self, name, age):
4         self.name = name
5         self.age = age
6     # instance method
7     def sing(self, song):
8         return "{} sings {}".format(self.name, song)
9
10    def dance(self):
11        return "{} is now dancing".format(self.name)
12
13    # instantiate the object
14    blu = Parrot("Blu", 10)
15
16    # call our instance methods
17    print(blu.sing("Happy"))
18    print(blu.dance())
```

Program Code

```
Blu sings 'Happy'
Blu is now dancing
```

Program Output

In the above program, we define two methods i.e sing() and dance().

These are called instance methods because they are called on an instance object i.e blu.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Inheritance

Inheritance is a way of creating a new class for using details of an existing class without modifying it.

The newly formed class is a derived class (or child class). Similarly, the existing class is a base class (or parent class).



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation


```

1 # parent class
2 class Bird:
3     def __init__(self):
4         print("Bird is ready")
5
6     def whoisThis(self):
7         print("Bird")
8
9     def swim(self):
10        print("Swim faster")
11 # child class
12 class Penguin(Bird):
13     def __init__(self):
14         # call super() function
15         super().__init__()
16         print("Penguin is ready")
17
18     def whoisThis(self):
19         print("Penguin")
20
21     def run(self):
22         print("Run faster")
23
24 peggy = Penguin()
25 peggy.whoisThis()
26 peggy.swim()
27 peggy.run()

```

Program Code

```

Bird is ready
Penguin is ready
Penguin
Swim faster
Run faster

```

Program Output

In the above program, we created two classes i.e. Bird (parent class) and Penguin (child class). The child class inherits the functions of parent class. We can see this from the swim() method.

Again, the child class modified the behavior of the parent class. We can see this from the whoisThis() method. Furthermore, we extend the functions of the parent class, by creating a new run() method.

Additionally, we use the super() function inside the __init__() method. This allows us to run the __init__() method of the parent class inside the child class.

Encapsulation

Using OOP in Python, we can restrict access to methods and variables.

This prevents data from direct modification which is called encapsulation.

In Python, we denote private attributes using underscore as the prefix i.e single `_` or double `__`.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

```

1 class Computer:
2     def __init__(self):
3         self.__maxprice = 900
4
5     def sell(self):
6         print("Selling Price: {}".format(self.__maxprice))
7
8     def setMaxPrice(self, price):
9         self.__maxprice = price
10
11 c = Computer()
12 c.sell()
13
14 # change the price
15 c.__maxprice = 1000
16 c.sell()
17
18 # using setter function
19 c.setMaxPrice(1000)
20 c.sell()

```

Program Code

```

Selling Price: 900
Selling Price: 900
Selling Price: 1000

```

Program Output

In the above program, we defined a Computer class.

We used `__init__()` method to store the maximum selling price of Computer. We tried to modify the price. However, we can't change it because Python treats the `__maxprice` as private attributes.

As shown, to change the value, we have to use a setter function i.e `setMaxPrice()` which takes price as a parameter.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Polymorphism

Polymorphism is an ability (in OOP) to use a common interface for multiple forms (data types).

Suppose, we need to color a shape, there are multiple shape options (rectangle, square, circle). However we could use the same method to color any shape.

This concept is called Polymorphism.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

```
1 class Parrot:
2     def fly(self):
3         print("Parrot can fly")
4
5     def swim(self):
6         print("Parrot can't swim")
7
8 class Penguin:
9
10    def fly(self):
11        print("Penguin can't fly")
12    def swim(self):
13        print("Penguin can swim")
14
15 # common interface
16 def flying_test(bird):
17     bird.fly()
18
19 #instantiate objects
20 blu = Parrot()
21 peggy = Penguin()
22
23 # passing the object
24 flying_test(blu)
25 flying_test(peggy)
```

Program Code

```
Parrot can fly
Penguin can't fly
```

Program Output

In the above program, we defined two classes Parrot and Penguin. Each of them have a common fly() method. However, their functions are different.

To use polymorphism, we created a common interface i.e flying_test() function that takes any object and calls the object's fly() method. Thus, when we passed the blu and peggy objects in the flying_test() function, it ran effectively.

Technology Driven by Innovation

Key Points to Remember:

- Object-Oriented Programming makes the program easy to understand as well as efficient.
- Since the class is sharable, the code can be reused.
- Data is safe and secure with data abstraction.
- Polymorphism allows the same interface for different objects, so programmers can write efficient code.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Objects and Classes



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Objects and Classes

Python is an object oriented programming language. Unlike procedure oriented programming, where the main emphasis is on functions, object oriented programming stresses on objects.

An object is simply a collection of data (variables) and methods (functions) that act on those data. Similarly, a class is a blueprint for that object.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

We can think of class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows etc. Based on these descriptions we build the house. House is the object.

As many houses can be made from a house's blueprint, we can create many objects from a class. An object is also called an instance of a class and the process of creating this object is called **instantiation**.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Defining a Class in Python

Like function definitions begin with the `def` keyword in Python, class definitions begin with a `class` keyword.

The first string inside the class is called docstring and has a brief description about the class. Although not mandatory, this is highly recommended.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Here is a simple class definition.

```
class MyNewClass:  
    '''This is a docstring. I have created a new class'''  
    pass
```

```
1 class Person:  
2     "This is a person class"  
3     age = 10  
4  
5     def greet(self):  
6         print('Hello')  
7  
8 # Output: 10  
9 print(Person.age)  
10  
11  
12 # Output: 'This is my second class'  
13 print(Person.__doc__)
```

Program Code

10
This is a person class

Program Output

A class creates a new local **namespace** where all its attributes are defined. Attributes may be data or functions.

There are also special attributes in it that begins with double underscores `__`. For example, `__doc__` gives us the docstring of that class.

As soon as we define a class, a new class object is created with the same name.

This class object allows us to access the different attributes as well as to instantiate new objects of that class.



Creating an Object in Python

We saw that the class object could be used to access different attributes.

It can also be used to create new object instances (instantiation) of that class. The procedure to create an object is similar to a [function](#) call.

```
>>> hadji = Person()
```



This will create a new object instance named hadji. We can access the attributes of objects using the object name prefix.

Attributes may be data or method. Methods of an object are corresponding functions of that class.

This means to say, since Person.greet is a function object (attribute of class), Person.greet will be a method object.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation


```
1 class Person:
2     "This is a person class"
3     age = 10
4     def greet(self):
5         print('Hello')
6
7 # create a new object of Person class
8 hadji = Person()
9
10 # Output: <function Person.greet>
11 print(Person.greet)
12
13 # Output: <bound method Person.greet of <__main__.Person object>>
14 print(hadji.greet)
15
16 # Calling object's greet() method
17 # Output: Hello
18 hadji.greet()
```

<function Person.greet at 0x7f8ec0a61550>
<bound method Person.greet of <__main__.Person object at 0x7f8ec0967c10>>
Hello

Program Code

Program Output

You may have noticed the self parameter in function definition inside the class but we called the method simply as hadji.greet() without any arguments. It still worked.

This is because, whenever an object calls its method, the object itself is passed as the first argument. So, hadji.greet() translates into Person.greet(hadji).

In general, calling a method with a list of n arguments is equivalent to calling the corresponding function with an argument list that is created by inserting the method's object before the first argument.

For these reasons, the first argument of the function in class must be the object itself. This is conventionally called self. It can be named otherwise but we highly recommend to follow the convention.

Now you must be familiar with class object, instance object, function object, method object and their differences.

Constructors in Python

Class functions that begin with double underscore `__` are called special functions as they have special meaning.

Of one particular interest is the `__init__()` function. This special function gets called whenever a new object of that class is instantiated.

This type of function is also called constructors in Object Oriented Programming (OOP). We normally use it to initialize all the variables.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

```

1 class ComplexNumber:
2     def __init__(self, r=0, i=0):
3         self.real = r
4         self.imag = i
5
6     def get_data(self):
7         print(f'{self.real}+{self.imag}j')
8
9 # Create a new ComplexNumber object
10 num1 = ComplexNumber(2, 3)
11
12 # Call get_data() method
13 # Output: 2+3j
14 num1.get_data()
15
16 # Create another ComplexNumber object
17 # and create a new attribute 'attr'
18 num2 = ComplexNumber(5)
19 num2.attr = 10
20
21 # Output: (5, 0, 10)|
22 print((num2.real, num2.imag, num2.attr))
23

```

Program Code

In the above example, we defined a new class to represent complex numbers. It has two functions, `__init__()` to initialize the variables (defaults to zero) and `get_data()` to display the number properly.

An interesting thing to note in the above step is that attributes of an object can be created on the fly. We created a new attribute `attr` for object `num2` and read it as well. But this does not create that attribute for object `num1`.

```

2+3j
(5, 0, 10)
Traceback (most recent call last):
  File "Untitled 5.py", line 26, in <module>
    print(num1.attr)
AttributeError: 'ComplexNumber' object has no attribute 'attr'

```

Program Output



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Deleting Attributes and Objects

Any attribute of an object can be deleted anytime, using the del statement. Try the following on the Python shell to see the output.

```
>>> num1 = ComplexNumber(2,3)
```

```
>>> del num1.imag
```

```
>>> num1.get_data() Traceback (most recent call last): ...
```

```
AttributeError: 'ComplexNumber' object has no attribute 'imag'
```

```
>>> del ComplexNumber.get_data
```

```
>>> num1.get_data() Traceback (most recent call last): ...
```

```
AttributeError: 'ComplexNumber' object has no attribute 'get_data'
```



We can even delete the object itself, using the del statement.

```
>>> c1 = ComplexNumber(1,3)
```

```
>>> del c1
```

```
>>> c1 Traceback (most recent call last): ... NameError:  
name 'c1' is not defined
```



FEU ALABANG



FEU DILIMAN



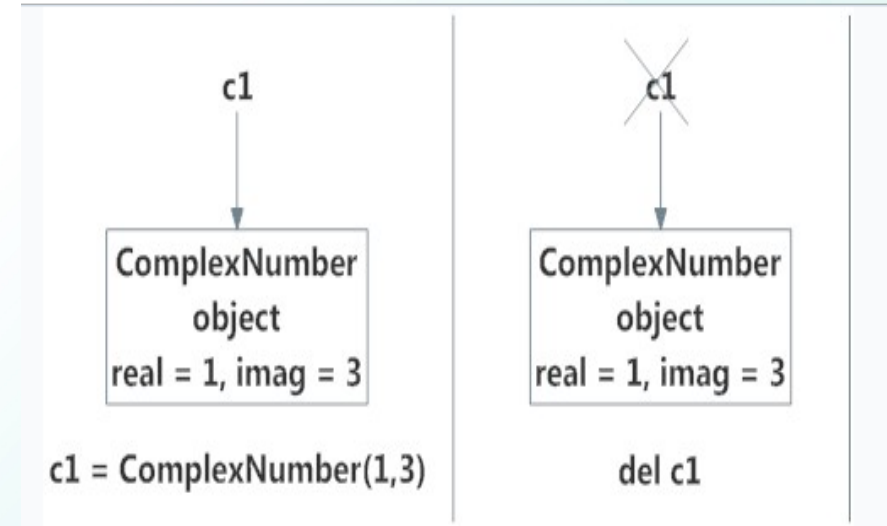
FEU TECH

Technology Driven by Innovation

Actually, it is more complicated than that. When we do `c1 = ComplexNumber(1,3)`, a new instance object is created in memory and the name `c1` binds with it.

On the command `del c1`, this binding is removed and the name `c1` is deleted from the corresponding namespace. The object however continues to exist in memory and if no other name is bound to it, it is later automatically destroyed.

This automatic destruction of unreferenced objects in Python is also called garbage collection.



Inheritance in Python



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Inheritance in Python

Inheritance is a powerful feature in object oriented programming.

It refers to defining a new class with little or no modification to an existing class.

The new class is called **derived (or child) class** and the one from which it inherits is called the **base (or parent) class**.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Inheritance Syntax

Derived class inherits features from the base class where new features can be added to it. This results in re-usability of code.

```
class BaseClass:  
    Body of base class  
class DerivedClass(BaseClass):  
    Body of derived class
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Example of Inheritance in Python

To demonstrate the use of inheritance, let us take an example.

A polygon is a closed figure with 3 or more sides. Say, we have a class called Polygon defined as follows.

```
1 class Polygon:
2     def __init__(self, no_of_sides):
3         self.n = no_of_sides
4         self.sides = [0 for i in range(no_of_sides)]
5
6     def inputSides(self):
7         self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in range(self.n)]
8
9     def dispSides(self):
10        for i in range(self.n):
11            print("Side",i+1,"is",self.sides[i])
```



This class has data attributes to store the number of sides n and magnitude of each side as a list called sides.

The inputSides() method takes in the magnitude of each side and dispSides() displays these side lengths.

A triangle is a polygon with 3 sides. So, we can create a class called Triangle which inherits from Polygon. This makes all the attributes of Polygon class available to the Triangle class.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

```

1 class Polygon:
2     def __init__(self, no_of_sides):
3         self.n = no_of_sides
4         self.sides = [0 for i in range(no_of_sides)]
5     def inputSides(self):
6         self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in range(self.n)]
7     def dispSides(self):
8         for i in range(self.n):
9             print("Side",i+1,"is",self.sides[i])
10 class Triangle(Polygon):
11     def __init__(self):
12         Polygon.__init__(self,3)
13     def findArea(self):
14         a, b, c = self.sides
15         # calculate the semi-perimeter
16         s = (a + b + c) / 2
17         area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
18         print('The area of the triangle is %0.2f' %area)
19
20 t = Triangle()
21 t.inputSides()
22 t.dispSides()
23 t.findArea()

```

Program Code

We can see that even though we did not define methods like inputSides() or dispSides() for class Triangle separately, we were able to use them.

If an attribute is not found in the class itself, the search continues to the base class. This repeats recursively, if the base class is itself derived from other classes.

```

Enter side 2 : 5
Enter side 3 : 4
Side 1 is 3.0
Side 2 is 5.0
Side 3 is 4.0
The area of the triangle is 6.00

```

Program Output



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Method Overriding in Python

In the above example, notice that `__init__()` method was defined in both classes, Triangle as well Polygon. When this happens, the method in the derived class overrides that in the base class. This is to say, `__init__()` in Triangle gets preference over the `__init__` in Polygon.

Generally when overriding a base method, we tend to extend the definition rather than simply replace it. The same is being done by calling the method in base class from the one in derived class (calling `Polygon.__init__()` from `__init__()` in Triangle).



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

A better option would be to use the built-in function `super()`. So, `super().__init__(3)` is equivalent to `Polygon.__init__(self,3)` and is preferred.

Two built-in functions `isinstance()` and `issubclass()` are used to check inheritances.

The function `isinstance()` returns `True` if the object is an instance of the class or other classes derived from it. Each and every class in Python inherits from the base class object.



Python Multiple Inheritance



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Multiple Inheritance

A class can be derived from more than one base class in Python, similar to C++. This is called multiple inheritance.

multiple inheritance, the features of all the base classes are inherited into the derived class. The syntax for multiple inheritance is similar to single inheritance.



FEU ALABANG



FEU DILIMAN

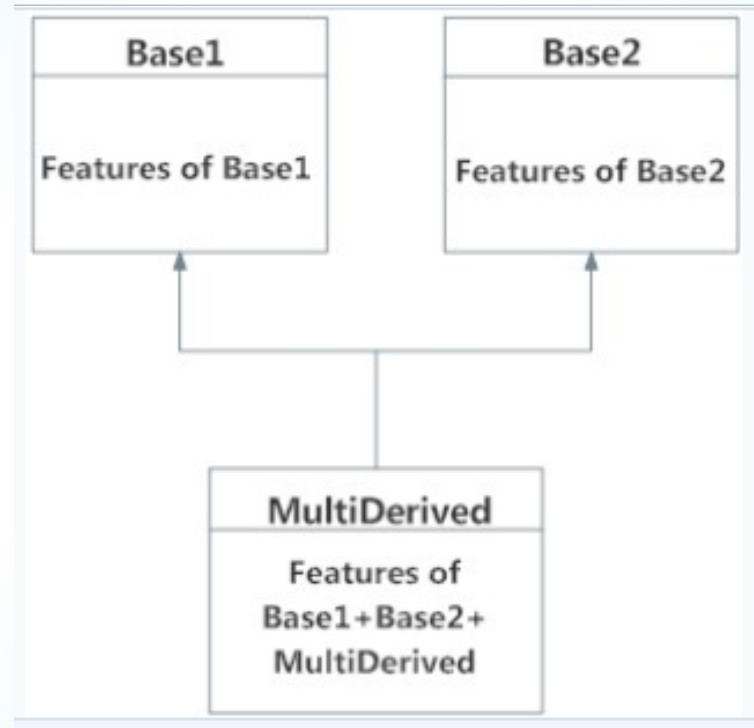


FEU TECH

Technology Driven by Innovation

Here, the MultiDerived class is derived from Base1 and Base2 classes.

```
class Base1:  
    pass  
  
class Base2:  
    pass  
  
class MultiDerived(Base1, Base2):  
    pass
```



The MultiDerived class inherits from both Base1 and Base2 classes.



Python Multilevel Inheritance

We can also inherit from a derived class. This is called multilevel inheritance. It can be of any depth in Python.

In multilevel inheritance, features of the base class and the derived class are inherited into the new derived class.



FEU ALABANG



FEU DILIMAN

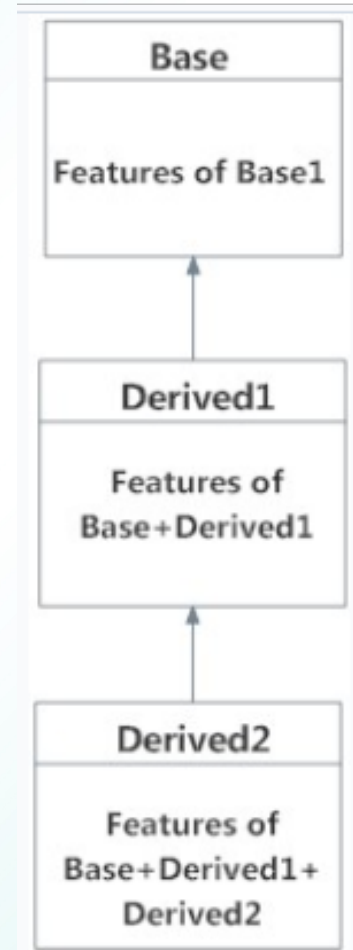


FEU TECH

Technology Driven by Innovation

Here, the Derived1 class is derived from the Base class, and the Derived2 class is derived from the Derived1 class.

```
class Base:  
    pass  
  
class Derived1(Base):  
    pass  
  
class Derived2(Derived1):  
    pass
```



Python Operator Overloading



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Python Operator Overloading

Python operators work for built-in classes. But the same operator behaves differently with different types. For example, the `+` operator will perform arithmetic addition on two numbers, merge two lists, or concatenate two strings.

This feature in Python that allows the same operator to have different meaning according to the context is called operator overloading.

So what happens when we use them with objects of a user-defined class? Let us consider the following class, which tries to simulate a point in 2-D coordinate system.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Example

```
1 class Point:
2     def __init__(self, x=0, y=0):
3         self.x = x
4         self.y = y
5
6 p1 = Point(1, 2)
7 p2 = Point(2, 3)
8 print(p1+p2)
```

```
Traceback (most recent call last):
  File "Untitled 6.py", line 9, in <module>
    print(p1+p2)
TypeError: unsupported operand type(s) for +: 'Point' and 'Point'
```

Here, we can see that a `TypeError` was raised, since Python didn't know how to add two `Point` objects together.

However, we can achieve this task in Python through operator overloading. But first, let's get a notion about special functions.



Python Special Functions

Class functions that begin with double underscore `__` are called special functions in Python.

These functions are not the typical functions that we define for a class. The `__init__()` function we defined above is one of them. It gets called every time we create a new object of that class.

Using special functions, we can make our class compatible with built-in functions.

```
>>> p1 = Point(2,3)
>>> print(p1) <__main__.Point object at 0x00000000031F8CC0>
```



FEU ALABANG



FEU DILIMAN

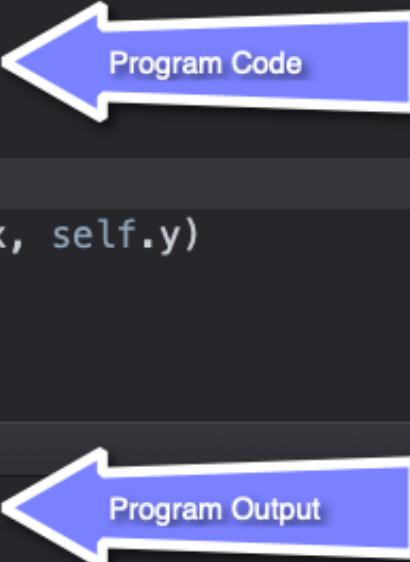


FEU TECH

Technology Driven by Innovation

```
1 class Point:
2     def __init__(self, x=0, y=0):
3         self.x = x
4         self.y = y
5
6     def __str__(self):
7         return "({0}, {1})".format(self.x, self.y)
8
9 p1 = Point(2, 3)
10 print(p1)
```

(2, 3)



Suppose we want the `print()` function to print the coordinates of the `Point` object instead of what we got.

We can define a `__str__()` method in our class that controls how the object gets printed. Let's look at how we can achieve this:

So, when you use `str(p1)` or `format(p1)`, Python internally calls the `p1.__str__()` method. Hence the name, special functions.

Overloading the + Operator

To overload the + operator, we will need to implement `__add__()` function in the class. With great power comes great responsibility.

We can do whatever we like, inside this function. But it is more sensible to return a Point object of the coordinate sum.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Example

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)
```

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x, self.y)

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)

p1 = Point(1, 2)
p2 = Point(2, 3)

print(p1+p2)
```

What actually happens is that, when you use `p1 + p2`, Python calls `p1.__add__(p2)` which in turn is `Point.__add__(p1,p2)`. After this, the addition operation is carried out the way we specified.

Output: (3,5)



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Operator Overloading

Operator	Expression	Internally
Addition	<code>p1 + p2</code>	<code>p1.__add__(p2)</code>
Subtraction	<code>p1 - p2</code>	<code>p1.__sub__(p2)</code>
Multiplication	<code>p1 * p2</code>	<code>p1.__mul__(p2)</code>
Power	<code>p1 ** p2</code>	<code>p1.__pow__(p2)</code>
Division	<code>p1 / p2</code>	<code>p1.__truediv__(p2)</code>
Floor Division	<code>p1 // p2</code>	<code>p1.__floordiv__(p2)</code>
Remainder (modulo)	<code>p1 % p2</code>	<code>p1.__mod__(p2)</code>
Bitwise Left Shift	<code>p1 << p2</code>	<code>p1.__lshift__(p2)</code>
Bitwise Right Shift	<code>p1 >> p2</code>	<code>p1.__rshift__(p2)</code>
Bitwise AND	<code>p1 & p2</code>	<code>p1.__and__(p2)</code>
Bitwise OR	<code>p1 p2</code>	<code>p1.__or__(p2)</code>
Bitwise XOR	<code>p1 ^ p2</code>	<code>p1.__xor__(p2)</code>
Bitwise NOT	<code>~p1</code>	<code>p1.__invert__()</code>

Technology Driven by Innovation

Overloading Comparison Operators

Python does not limit operator overloading to arithmetic operators only. We can overload comparison operators as well.

Suppose we wanted to implement the less than symbol `<` in our Point class.

Let us compare the magnitude of these points from the origin and return the result for this purpose. It can be implemented as follows.

Example

```
1 # overloading the less than operator
2 class Point:
3     def __init__(self, x=0, y=0):
4         self.x = x
5         self.y = y
6
7     def __str__(self):
8         return "{0},{1}".format(self.x, self.y)
9
10    def __lt__(self, other):
11        self_mag = (self.x ** 2) + (self.y ** 2)
12        other_mag = (other.x ** 2) + (other.y ** 2)
13        return self_mag < other_mag
14
15    p1 = Point(1,1)
16    p2 = Point(-2,-3)
17    p3 = Point(1,-1)
18
19    # use less than
20    print(p1<p2)
21    print(p2<p3)
22    print(p1<p3)
```

True
False
False

Similarly, the special functions that we need to implement, to overload other comparison operators are tabulated below.

Operator	Expression	Internally
Less than	<code>p1 < p2</code>	<code>p1.__lt__(p2)</code>
Less than or equal to	<code>p1 <= p2</code>	<code>p1.__le__(p2)</code>
Equal to	<code>p1 == p2</code>	<code>p1.__eq__(p2)</code>
Not equal to	<code>p1 != p2</code>	<code>p1.__ne__(p2)</code>
Greater than	<code>p1 > p2</code>	<code>p1.__gt__(p2)</code>
Greater than or equal to	<code>p1 >= p2</code>	<code>p1.__ge__(p2)</code>